

**SIMATS ENGINEERING**  
**ASSESSMENT TOOL 1: Constraint-Based Problem Solving**

**COURSE NAME: COMPUTER VISION COURSE CODE: ITA0510 Slot A**

**COURSE OUTCOME COVERED**

**CO5: Develop computer vision applications using OpenCV for performing recognition and analysis on images and videos. (BTL 6)**

**Assessment Tool Weightage: 40%**

**QUESTIONS:5 Total Marks 50 Duration :60 Minutes Annexure A**  
**Constraint-Based Problem Solving**

**Code of Conduct:**

***I P.Jyothika(192311156) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. IL= understand that any violation of academic integrity rules will result in disciplinary action.***

**Signature of the student:**

**Rubrics for Evaluation**

| Criteria                | Weightage (%) | Level 4 (Exemplary)                                                            | Level 3 (Proficient)                                  | Level 2 (Developing)                          | Level 1 (Beginning)                      | Marks(100) |
|-------------------------|---------------|--------------------------------------------------------------------------------|-------------------------------------------------------|-----------------------------------------------|------------------------------------------|------------|
| Problem Understanding   | 25%           | Clearly defines the problem and identifies all relevant constraints accurately | Identifies most constraints with minor gaps           | Partial understanding; misses key constraints | Fails to identify constraints correctly  |            |
| Constraint Analysis     | 25%           | Analyzes constraints effectively and prioritizes them logically                | Provides reasonable analysis with some prioritization | Limited analysis; weak prioritization         | No meaningful analysis or prioritization |            |
| Solution Development    | 25%           | Develops optimal and feasible solutions satisfying all constraints             | Develops workable solutions with minor limitations    | Solutions partially satisfy constraints       | Solutions do not meet constraints        |            |
| Application of Concepts | 15%           | Effectively applies appropriate concepts, methods, or tools                    | Applies concepts with minor errors                    | Limited application; requires guidance        | Unable to apply relevant concepts        |            |

|               |     |                                                                    |                                          |                               |                           |  |
|---------------|-----|--------------------------------------------------------------------|------------------------------------------|-------------------------------|---------------------------|--|
| Justification | 10% | Provides strong justification for decisions with logical reasoning | Provides justification with some clarity | Weak or unclear justification | No justification provided |  |
|---------------|-----|--------------------------------------------------------------------|------------------------------------------|-------------------------------|---------------------------|--|

Constraint-Based Problem Solving: Analyze the problem and design an end-to-end computer vision pipeline using OpenCV, clearly identifying each stage from input acquisition to final output. Ensure that your solution satisfies all given constraints and justify your design decisions at each stage.

### 1. Facial Expression Recognition for Mobile Application

A mobile app must detect facial expressions (happy, sad, neutral) with the following constraints: A. Limited processing power (mobile CPU)

B. Real-time performance

C. Robustness to slight head movement and lighting changes

Design a solution using OpenCV for face detection and expression analysis. Justify how your approach balances accuracy and efficiency.

### **ANSWER:**

### OpenCV Pipeline

Camera → Preprocessing → Face Detection → Face Alignment → Expression Analysis → Smoothing → Output

1. Input Acquisition
  - Capture live frames from the mobile camera using OpenCV.
  - Resize frames to a small size such as 320×240 to reduce processing.
2. Preprocessing
  - Convert the frame from RGB/BGR to grayscale.
  - Apply histogram equalization or CLAHE to handle small lighting changes.
  - Apply light Gaussian blur to reduce noise.
3. Face Detection
  - Use OpenCV Haar Cascade or LBP Cascade face detector.
  - Detect only the face region instead of processing the complete image.
4. Face Alignment
  - Estimate face position and crop the detected face.
  - Normalize the face size so that small head movements have less effect.
5. Expression Analysis
  - Extract simple facial features such as mouth shape, eye region, and eyebrows.
  - A lightweight classifier such as SVM/KNN can classify the face as Happy, Sad, or Neutral.
6. Temporal Smoothing
  - Analyze several consecutive frames and use the majority result.
  - This prevents sudden changes caused by one incorrect frame.
7. Output
  - Display the detected expression on the mobile screen using `cv2.putText()`.

## Constraint Justification

- Limited CPU: Small frames, grayscale images, and lightweight Haar/LBP detection reduce computation.
- Real-time: Only the detected face region is analyzed instead of the entire frame.
- Head movement: Face localization and normalization handle slight movement.
- Lighting: Histogram equalization/CLAHE improves robustness to moderate illumination changes.
- Accuracy vs efficiency: A lightweight feature-based classifier is faster than a large deep-learning model, making it suitable for a mobile CPU.

## 2. Smart Traffic Monitoring System

A city authority needs a system to:

- A. Detect vehicles from live video streams
- B. Track movement across frames
- C. Count vehicles and display statistics
- D. Operate continuously with minimal delay

Design a complete OpenCV-based system including video handling, detection, tracking, and visualization (drawing functions). Justify how your system ensures accuracy and real-time performance.

### **ANSWER:**

# Smart Traffic Monitoring System

Goal: Detect, track, count vehicles and continuously display traffic statistics with minimum delay.

## OpenCV Pipeline

Live Video → Frame Preprocessing → Vehicle Detection → Tracking → Counting → Visualization → Statistics

1. Video Acquisition
  - Read live CCTV/video using `cv2.VideoCapture()`.
  - Process frames continuously.
2. Preprocessing
  - Resize frames to a suitable resolution.
  - Convert to grayscale if required.
  - Apply Gaussian blur to reduce noise.
3. Vehicle Detection
  - Use background subtraction such as MOG2 for a fixed camera.
  - Obtain foreground regions containing moving vehicles.
  - Apply thresholding and morphological operations to remove small noise.
4. Contour Detection
  - Use `cv2.findContours()` to find vehicle regions.
  - Remove very small contours using an area threshold.
  - Draw bounding boxes around detected vehicles.

5. Vehicle Tracking
  - Calculate the center point of each detected vehicle.
  - Track vehicles between consecutive frames using a simple centroid-based tracker.
  - Assign a unique ID to each vehicle.
6. Vehicle Counting
  - Draw a virtual counting line using `cv2.line()`.
  - When a vehicle ID crosses the line, increase the vehicle count.
  - Because each vehicle has an ID, the same vehicle is not counted repeatedly.
7. Visualization
  - `cv2.rectangle()` → vehicle bounding boxes.
  - `cv2.putText()` → vehicle ID and count.
  - `cv2.line()` → counting line.
  - Display total vehicles and other statistics on the video.
8. Output
  - Show live video with detected vehicles, tracking IDs and vehicle count.

### Constraint Justification

- Live detection: Frames are processed continuously.
- Tracking: IDs prevent duplicate counting.
- Accuracy: Area filtering and morphological operations remove small noise.
- Real-time: Background subtraction is computationally cheaper than running a heavy detector on every frame.
- Minimal delay: Resize frames and process only necessary operations.

Conclusion: For a fixed CCTV camera, background subtraction + centroid tracking provides a good balance between speed, accuracy and computational cost.

### 3. Real-Time Object Detection on Mobile Device

A mobile application must:

- A. Detect objects using the camera
- B. Run in real time
- C. Use limited CPU and memory

Design an OpenCV-based pipeline from image acquisition to detection and display. Justify how your design ensures efficiency under constraints.

### ANSWER:

## Real-Time Object Detection on Mobile Device

Goal: Detect objects from a mobile camera while using limited CPU and memory.

### OpenCV Pipeline

Camera → Frame Resize → Preprocessing → Lightweight Object Detector → Bounding Boxes → Display

1. Image Acquisition

- Capture frames using the mobile camera.
- Use `cv2.VideoCapture()` or the mobile camera interface.
- 2. Frame Resizing
  - Resize the input to a smaller resolution such as 320×320.
  - This reduces memory usage and computation.
- 3. Preprocessing
  - Normalize pixel values.
  - Convert the image into the format required by the detector.
- 4. Object Detection
  - Use a lightweight model such as MobileNet-SSD with OpenCV DNN.
  - Load the model using `cv2.dnn.readNet()`.
  - Perform inference on each frame.
- 5. Confidence Filtering
  - Calculate the confidence score for each detected object.
  - Display only detections above a threshold, for example 0.5.
  - This reduces false detections.
- 6. Drawing Results
  - Use `cv2.rectangle()` to draw bounding boxes.
  - Use `cv2.putText()` to display object names and confidence.
- 7. Output
  - Display the processed frame continuously on the mobile screen.

### Constraint Justification

- Limited CPU: MobileNet-SSD is lighter than large object-detection networks.
- Limited memory: Small input frames and a compact model reduce memory usage.
- Real-time: Resize images and avoid unnecessary processing.
- Accuracy: Confidence thresholding removes weak detections.
- Efficiency: OpenCV DNN provides optimized inference.

Conclusion: A lightweight detector with reduced resolution provides a practical balance between detection accuracy and real-time mobile performance.

### 4. Smart Attendance System using Face Recognition

A system must:

- A. Capture faces from a webcam
- B. Recognize registered users
- C. Mark attendance automatically

Design a complete pipeline using OpenCV. Justify each stage and how the system avoids false recognition.

**ANSWER:**

## Smart Attendance System Using Face Recognition

**Goal:** Automatically identify registered students and mark their attendance without false recognition.

## OpenCV Pipeline

**Webcam → Face Detection → Preprocessing → Face Recognition → Confidence Check → Attendance → Display**

1. **Image Acquisition**
  - Capture live video using a webcam with `cv2.VideoCapture()`.
2. **Face Detection**
  - Use OpenCV Haar Cascade/LBP Cascade to detect faces.
  - Extract only the face region.
3. **Preprocessing**
  - Convert the face to grayscale.
  - Resize all faces to the same size.
  - Normalize illumination to improve recognition.
4. **Face Database**
  - Store multiple images of each registered person.
  - Assign a unique ID/name to each person.
5. **Face Recognition**
  - Train a lightweight OpenCV recognizer such as **LBPH (Local Binary Patterns Histograms)**.
  - Compare the detected face with registered faces.
6. **Confidence/Threshold Check**
  - Accept the identity only if the recognition confidence is within the predefined threshold.
  - Otherwise label the face as **Unknown**.
7. **Multiple-Frame Verification**
  - Require the same identity to be detected across several consecutive frames.
  - This reduces false recognition caused by temporary noise or poor lighting.
8. **Attendance Marking**
  - When a registered person is confidently identified, record:
    - Name/ID
    - Date
    - Time
  - Check whether attendance is already marked to avoid duplicate entries.
9. **Output**
  - Display the person's name and attendance status using `cv2.putText()`.

## Constraint Justification

- **False recognition:** Confidence threshold + multiple-frame confirmation reduce incorrect matches.
- **Lighting variation:** Grayscale and normalization improve recognition.
- **Efficiency:** LBPH is relatively lightweight and suitable for a basic computer.
- **Duplicate attendance:** Maintain a list/database of already marked users.
- **Reliability:** Multiple images per registered person improve recognition under different face conditions.

**Conclusion:** The combination of **face detection + LBPH recognition + confidence threshold + multi-frame verification** provides a simple and reliable attendance system.

## 5. Motion Detection for Home Security

A home security system must:

- A. Detect motion from a camera feed
- B. Trigger alerts only for significant movement
- C. Reduce false alarms

Design the pipeline and justify how your approach handles noise and background variations.

**ANSWER:**

# Motion Detection for Home Security

Goal: Detect significant movement while reducing false alarms caused by noise and background changes.

## OpenCV Pipeline

Camera → Frame Acquisition → Grayscale → Blur → Background Comparison → Threshold → Morphology → Contours → Motion Decision → Alert

1. Camera Input
  - Capture continuous frames using `cv2.VideoCapture()`.
2. Grayscale Conversion
  - Convert frames to grayscale.
  - This reduces computation because color information is unnecessary for basic motion detection.
3. Noise Reduction
  - Apply Gaussian blur to remove small camera noise and insignificant changes.
4. Background Modeling
  - Use MOG2 background subtraction to separate moving objects from the background.
5. Thresholding
  - Apply a binary threshold to identify significant changes.
6. Morphological Processing
  - Apply opening/closing operations to remove small isolated noise and fill small gaps.
7. Contour Detection
  - Use `cv2.findContours()` to identify moving regions.
  - Ignore contours smaller than a predefined area.
8. Significant Motion Decision
  - Trigger an alert only when:
    - The moving area is larger than the minimum threshold, and
    - Motion is detected for multiple consecutive frames.
9. Alert
  - Display "**MOTION DETECTED**" using `cv2.putText()`.
  - The system can then activate a buzzer, notification, or other external alert mechanism.

## Constraint Justification

- Noise reduction: Gaussian blur removes tiny pixel changes.
- Background variations: MOG2 adapts to gradual background changes such as lighting variations.
- False alarms: Minimum contour area prevents alerts from small movements such as shadows or sensor noise.
- Significant movement: Consecutive-frame confirmation prevents one-frame false alarms.
- Real-time: Grayscale processing and background subtraction are computationally inexpensive.

Conclusion: MOG2 + thresholding + morphology + contour-area filtering + consecutive-frame confirmation provides an efficient and reliable motion detection system for home security.